Amazon Appliances Review Analytics

Course: Introduction to Big Data
Team: Team 34
Repository: [github]
Cluster path: /home/team34/projects/amazon-reviews-bigdata

Authors:

Role	Name	Email
Data engineer	*Dmitriy Ryazanov*	*d.ryazanov@innopolis.university*
ML Specialist	*Ivan Makarov*	*i.makarov@innopolis.university*
Data scientist	*Maksim Ilin*	*m.ilin@innopolis.university*

Date: May 2026

1. Introduction

This project implements an end-to-end batch pipeline for the **Amazon Reviews 2023 — Appliances** slice from the McAuley-Lab dataset on Hugging Face. The goal is to ingest large-scale JSON Lines, land the data in a **PostgreSQL** relational warehouse, export it to **HDFS** in columnar form via **Apache Sqoop** (MapReduce on YARN), and prepare downstream analytics (Hive EDA, Spark ML, and **intended** Superset dashboard — **§9.3** for access limitations in our delivery).

1.1 Business objectives

- Understand **customer satisfaction signals** from review text, ratings, and verified purchase flags for the Appliances category
- Relate reviews to **product metadata** (price, categories, store) to support pricing and assortment insights
- Prepare a **large, analysis-ready** dataset suitable for distributed SQL, Hive EDA, and multiclass rating prediction in Spark MLlib
- Deliver **reproducible** pipeline runs and a **single public** Superset dashboard for stakeholders and grading — **blocked in our run** by Superset credential access; see **§9.3**

2. Data description

Source: McAuley-Lab / Amazon-Reviews-2023 — raw **JSON Lines**:

- Appliances.jsonl — customer reviews.
- meta_Appliances.jsonl — product metadata keyed by parent_asin.

Staging (CSV): generated under data/staging/:

- metadata.csv — one row per product (parent_asin primary key in the relational model).
- reviews.csv — one row per review after filtering to products present in metadata and **deduplication** on (parent_asin, review_id).

Relational warehouse (PostgreSQL): tables metadata and reviews with a foreign key from reviews to metadata

Data lake (HDFS): after Sqoop:

- Parquet + Snappy under ${HDFS_WAREHOUSE_BASE}/reviews and …/metadata.
- Staging CSV copies under …/staging/ for Hive/external table options

3. Data characteristics

Aspect	**Approximate / typical values (full Appliances split)**
Raw review JSONL lines	~2.13M (before dedup)
Staged review rows	~2.11M (after dedup)
Metadata rows	~94k unique parent_asin
Raw on-disk size	~1.1 GiB combined JSONL (order of magnitude)
Time feature	review_timestamp (TIMESTAMPTZ) — satisfies datetime requirement
Feature richness	rating, text, price, categories JSON, store, helpful votes, verified purchase, images JSON — **>6** explanatory fields for ML/EDA

4. Architecture of the data pipeline

High-level flow:

JSONL (Hugging Face)
→ fetch + transform (Python)
→ validate (Python)
→ PostgreSQL (migrations + COPY)
→ Sqoop import (JDBC → HDFS Parquet, Snappy, MR on YARN)
→ optional HDFS staging CSV copy
→ Stage 2: Hive external / managed tables + HiveQL EDA (Beeline, Tez)
→ Stage 3: Spark ML on YARN (read Hive / HDFS)
→ Stage 4: Apache Superset (public dashboard when credentials work — **our run: §9.3**)

Orchestration:

- **Stage 1:** scripts/stage1.sh — collection, storage, validation, Sqoop→HDFS
- **Stage 2:** scripts/stage2.sh — Hive DDL + EDA via beeline, outputs output/hive_results.txt
- **Stage 3 / 4:** scripts/stage3.sh, scripts/stage4.sh — Spark and dashboard prep

Configuration: .env, secrets/.psql.pass, optional HIVE_PASSWORD, HDFS_WAREHOUSE_BASE, HDFS_REPLICATION

4.1 PostgreSQL → HDFS: file format and compression (Parquet + Snappy)

Decision	Choice	Rationale
File format	**Parquet** (--as-parquetfile)	**Columnar** storage matches Stage 2–4 workloads: Hive and Spark overwhelmingly run **aggregations, filters, and joins** over **subsets of columns**. Parquet lets engines read only the columns a query needs and use **per-column statistics** to skip data, which reduces I/O versus **row-oriented** options such as Avro for this analytic use case.

Decision	Choice	Rationale
Compression	**Snappy** (--compression-codec snappy)	**Snappy** favors **fast compression and decompression** over the last percent of disk savings. For multi-million-row tables, **Gzip** often shrinks files slightly more but **increases CPU time** on full scans; **Bzip2** improves ratio further but is **too slow** for iterative EDA and ML. Snappy therefore gives a **strong reduction** versus uncompressed Parquet while keeping **retrieval latency** predictable for graders and for repeated pipeline runs

Alternatives considered: Avro good for row-wise integration and schema evolution but is a weaker default when the primary consumer is **SQL analytics** over static dumps. **Gzip / Bzip2** remain valid when minimizing disk is paramount and CPU is abundant; for this project, **query turnaround** and alignment with Spark/Hive defaults led to **Parquet + Snappy**

5. Input and output for each stage

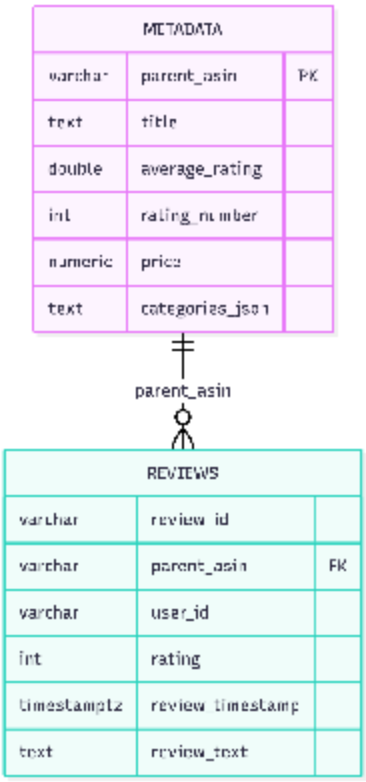
Stage	Input	Main activities	Output
I	JSONL URLs; optional env limits	Download/stream; CSV staging; validate; migrate + load; Sqoop; HDFS CSV upload	data/raw/*.jsonl, data/staging/*.csv, Postgres tables, **HDFS columnar export (Parquet + Snappy)** + staging/CSV
II	HDFS Parquet/CSV from Stage I	Hive DB + external/managed tables; partitioning/bucketing where designed; HiveQL EDA	Hive tables, output/hive_results.txt, query extracts for Superset
III	Hive / HDFS curated tables	PySpark MLlib; --master yarn; grid search + CV; metrics	Model artifacts, prediction tables on HDFS/Hive
IV	Stage II–III outputs	Superset datasets/charts; publish **one** public dashboard	**No public URL** — Superset login failed; TA notified **~1 h before presentation** (**§9.3**). Figures + `output/` substitute for the dashboard

6. Data preparation

6.1 Relational model and ER diagram

Entities:

- **metadata** — product grain (parent_asin PK)
- **reviews** — review grain; composite uniqueness (parent_asin, review_id); FK to metadata(parent_asin)



6.2 Samples from the database

SELECT parent_asin, title, average_rating, price

FROM metadata

ORDER BY rating_number DESC NULLS LAST

LIMIT 5;

SELECT review_id, parent_asin, rating, review_timestamp

FROM reviews

ORDER BY review_timestamp DESC

LIMIT 5;

6.3 Hive tables and preparation for analysis

Purpose. After Stage 1, **Parquet + Snappy** for reviews and metadata already lives under **${HDFS_WAREHOUSE_BASE}** on HDFS. Stage 2 registers that data in **Hive**, materializes **partitioned / bucketed** tables for efficient scans, runs a fixed set of **EDA queries**, and writes **output/q1.csv … output/q5.csv** plus **output/hive_results.txt** for the report and Superset.

Entry point. bash scripts/stage2.sh. The script requires **beeline** on PATH, a reachable **HiveServer2** JDBC URL (default in repo: jdbc:hive2://hadoop-03.uni.innopolis.ru:10001/default), and **HIVE_PASSWORD** (and optionally **HIVE_USER**) in the environment so Beeline never prompts interactively. Other important variables (see scripts/stage2.sh and .env):

Variable	**Role (typical)**
HIVE_DB_NAME	Logical Hive database, default team34_projectdb
HIVE_DB_LOCATION	HDFS directory for the DB warehouse, e.g. /user/team34/project/hive/warehouse/team34_projectdb
HDFS_WAREHOUSE_BASE	Where Sqoop wrote **reviews/** and **metadata/** Parquet trees
HIVE_REVIEWS_OPTIMIZED_PATH	Target path for curated reviews table (default under HIVE_DB_LOCATION)
HIVE_METADATA_BUCKETED_PATH	Target path for curated metadata table

Step 1 — sql/stage2_hive_init.hql (via Beeline). Hive session flags turn on **dynamic partitioning**, **bucketing**, and cap partition counts; **vectorized execution** is disabled for Parquet in this script for compatibility with the course stack.

1. **CREATE DATABASE** / **USE** ${hiveconf:hive_db_name} at ${hiveconf:hive_db_location}.

2. **reviews_ext** — **EXTERNAL** table, **STORED AS PARQUET**, location **${hdfs_warehouse_base}/reviews** (Sqoop export).

3. **metadata_ext** — same for **${hdfs_warehouse_base}/metadata**.

4. **reviews_optimized** — **EXTERNAL**, **PARTITIONED BY** (review_year, review_month), **CLUSTERED BY** (parent_asin) **INTO 8 BUCKETS**, **STORED AS PARQUET** at ${hive_reviews_optimized_path}. Data is filled with **INSERT OVERWRITE … PARTITION**: review_timestamp is cast from Unix time, treating large numeric values as **milliseconds** vs seconds.

5. **metadata_bucketed** — **EXTERNAL**, **CLUSTERED BY** (parent_asin) **INTO 8 BUCKETS**; **price** is cleaned from the string column and cast to **DECIMAL(12,2)**.

6. **DROP reviews_ext** and **metadata_ext** so routine EDA uses only the optimized tables.

7. **SHOW TABLES** for a quick sanity check.

Step 2 — EDA queries sql/q1.hql … sql/q5.hql. For each qN, the script builds a temporary Beeline script that **USEs** the database, **DROPs** qN_results if present, **CREATE TABLE qN_results STORED AS PARQUET AS** + the query body, then **SELECT *** into **output/qN.csv** (CSV2 with header). The log appends timings and a final **SHOW TABLES LIKE 'q*_results'**.

File	What it computes (high level)
q1.hql	By **main_category** and **verified vs not verified**: counts, **avg rating**, **avg helpful_vote**, **low_rating_share** (rating ≤ 2); groups with ≥ 500 reviews.
q2.hql	By **review_year**, **review_month**: volume, avg rating, low-rating share, verified ratio, avg helpful votes — **time series** for the dashboard.

File	What it computes (high level)
q3.hql	By **store** (≥ 200 reviews): ranking stores by **low_rating_share** (and avg rating, volume); **LIMIT 20**.
q4.hql	By **price band** (null / 0–25 / 25–75 / … / 300+): volume and quality proxies — links list price to ratings.
q5.hql	By **main_category**: low-rating counts, shares, helpful votes on low ratings; **LIMIT 20** categories with ≥ 500 reviews.

Alternative runner. python scripts/stage2_spark_eda.py --mode all runs the same **stage2_hive_init.hql** logic through **Spark SQL** with Hive support and exports **q1–q3** to CSV by default (see script for --mode and query list). Useful for laptops with Spark but **without** Beeline; it still needs a metastore/HDFS layout consistent with **config.constants.HDFS_WAREHOUSE_BASE**.

Local run note. On a developer machine without **Hadoop client + Beeline + VPN** to the IU cluster, scripts/stage2.sh exits immediately with **beeline is required for Stage 2**. The team runs Stage 2 **on the cluster login node** (or any host where beeline and network access to HiveServer2 are configured) after Stage 1 has populated HDFS.

7. Data analysis (EDA)

7.1 Approach

- **Primary engine: HiveQL** executed with **Beeline** against **HiveServer2** on the course cluster (execution engine as configured server-side, e.g. Tez).
- **Artifacts:** five analytical extracts **q1–q5** (see §7.3) materialized as **q*_results** tables and **output/q*.csv**.
- **Optional: scripts/stage2_spark_eda.py** for Spark-based execution of init + a subset of queries where Beeline is unavailable.

7.2 Example analysis themes (mapped to Stage 2 SQL)

The shipped HiveQL covers the following themes (use **q*.csv** and Superset for visuals):

1. **Category × verified purchase** — quality and volume (q1.hql).
2. **Trends over time** — monthly panels (q2.hql).
3. **Store-level stress** — high **low_rating_share** (q3.hql).
4. **Price bands vs ratings** (q4.hql).
5. **Categories with concentrated low ratings and helpful votes** (q5.hql).
6. **Dashboard extras** — e.g. text length vs helpful votes can be added as a follow-up query.

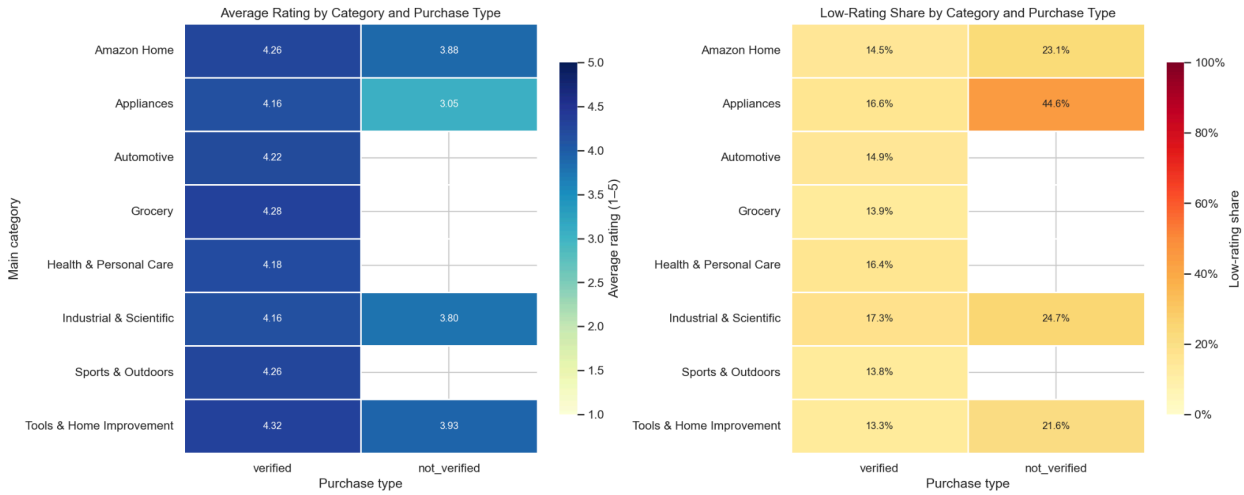
7.3 Charts and interpretation

Stage 2 produces five Hive extracts (`output/q1.csv` … `output/q5.csv`). The `notebooks/eda_stage2.ipynb` notebook builds seaborn/matplotlib charts from those files.

7.3.1 Terminology (Stage 2 metrics)

Term	Meaning
`main_category`	Coarse product department taken from metadata (used in q1, q5).
`purchase_type`	`verified` if Amazon marks the review as a verified purchase; `not_verified` otherwise (q1).
`review_count`	Number of reviews counted in the aggregation bucket (volume / exposure).
`avg_rating`	Mean star rating on the usual **1–5** scale within the bucket.
`low_rating_share`	Share of reviews with rating **≤ 2** (computed in SQL as an average of 0/1 indicators). Treat as a **dissatisfaction / risk** signal alongside `avg_rating`.
`avg_helpful_vote`	Mean “helpful” vote count per review in the bucket (q1, q3).
`review_year` / `review_month`	Calendar parts of `review_timestamp` used to build a monthly time axis (q2).
`verified_purchase_ratio`	Among reviews in that month, the fraction that are verified purchases (q2).
`store`	Seller / storefront identifier from metadata (q3).
`price_band`	Bucketed list price (e.g. `0_25`, `25_75`, …, `unknown`) for linking price level to ratings (q4).
`avg_helpful_on_low_ratings`	Average helpful votes **restricted to low-rating reviews** (1–2★) in that category — engagement with negative content (q5).

7.3.2 Q1 — Category × verified purchase

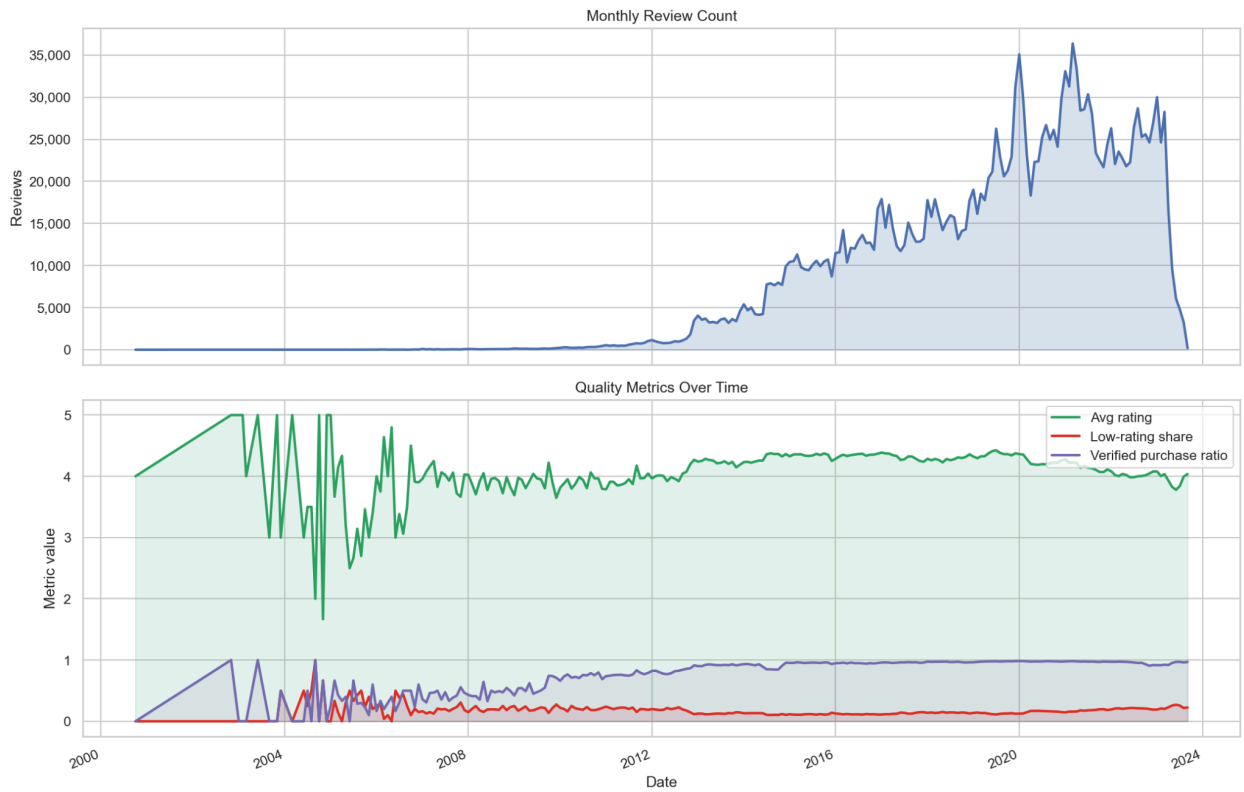


How to read the figure. Two heatmaps share the same row ordering: **top 12** categories by `review_count` with `unknown` removed. **Columns** are `verified` vs `not_verified`. The left panel colours `avg_rating` (darker blue → higher mean stars). The right panel colours `low_rating_share` (darker red → more 1–2★ reviews).

Takeaways. Where both purchase types exist for a category, **verified** rows often show **higher** `avg_rating` and **lower** `low_rating_share` than **not_verified** — consistent with Amazon-attested purchases correlating with fewer extreme negatives in this slice. Large categories (e.g. **Tools & Home Improvement**, **Amazon Home**, **Appliances**) dominate volume; **Appliances / not_verified** can show a wide gap versus **verified** on both panels, which supports treating verification status as a segmentation axis in dashboards — without claiming causality from observational data alone.

Business conclusion. Verified purchases in large categories often result in higher ratings and a lower proportion of low stars—this is the "real buyer" segment, which is safer for building satisfaction KPIs and category comparisons. For not_verified, there's a higher risk of "noise" and biased negative reviews. For communications with suppliers and complaint prioritization, it's better to filter or display verified vs. not separately.

7.3.3 Q2 — Monthly volume and quality trends



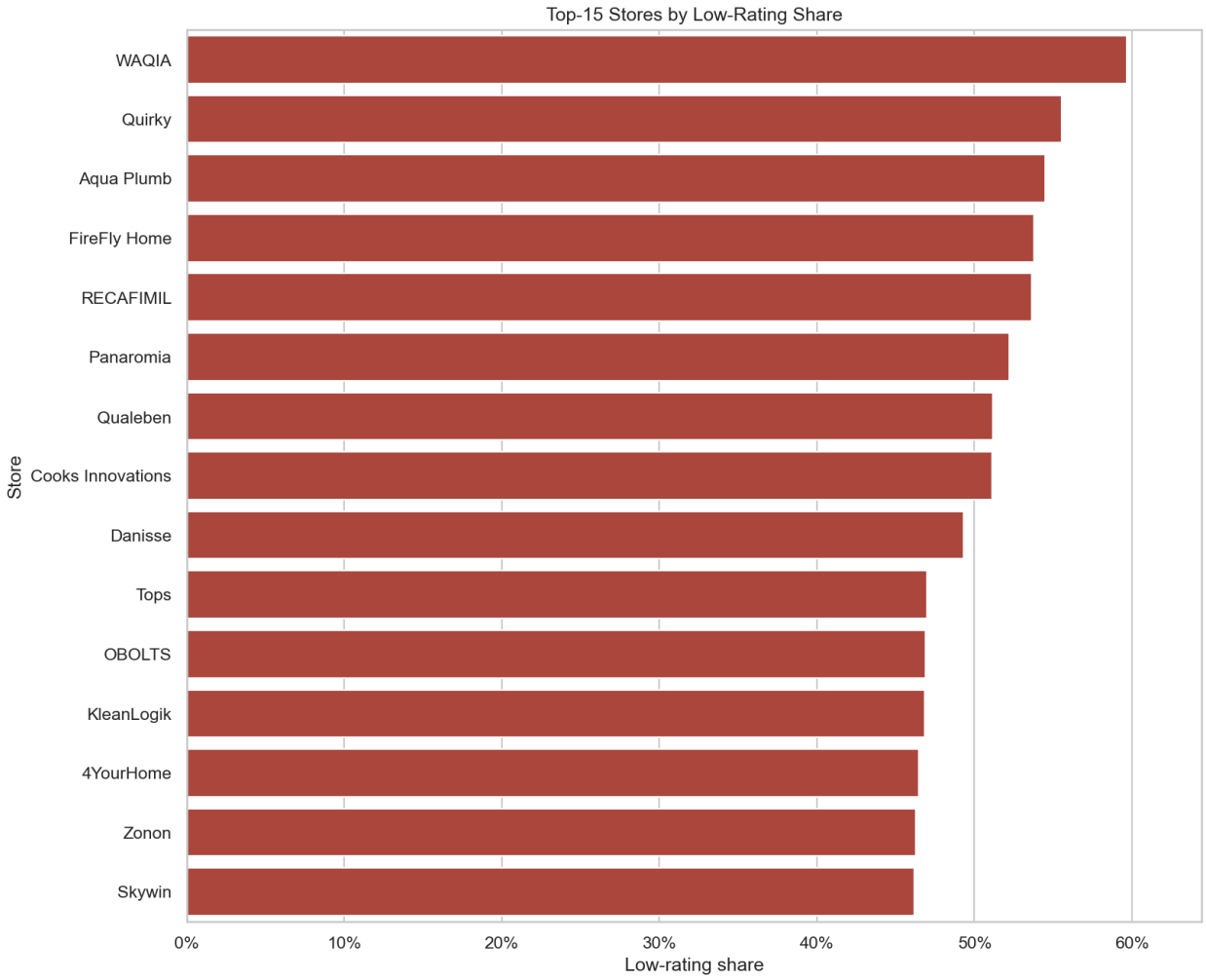
How to read the figure. The **upper** subplot is `review_count` by month (`review_year`–`review_month`). The **lower** subplot overlays `avg_rating`, `low_rating_share`, and `verified_purchase_ratio` on **one linear y-axis**; rating values (~3–5) therefore use most of the vertical span while the two ratio curves often sit nearer the bottom — interpret **direction and timing**, not vertical comparison between “rating” and “share” lines.

Takeaways. Peaks in `review_count` identify eras of highest activity; sharp drops at the **trailing edge** of the export window usually reflect **incomplete months or cutoff**, not necessarily a real collapse in demand. Rising `low_rating_share` alongside still-high `verified_purchase_ratio` late in the series suggests negativity concentrated among verified buyers while composition remains verified-heavy — useful for prioritising review moderation and product QA by period.

Business conclusion. Peak review volumes indicate periods of maximum "voice of the customer"—it's logical to associate these with launches, promotions, and supplier changes. A sharp drop on the right edge is more often an artifact of the data window than a collapse in demand—don't take trends over the last one or two points as fact without verifying the full month. If low ratings increase with a high verified share, this signals a genuine issue with the

product/fulfillment team, not bots. Action: Implement a monthly control tracker: volume + low_rating_share + verified share as an early indicator for QA and procurement.

7.3.4 Q3 — Stores ranked by low-rating share

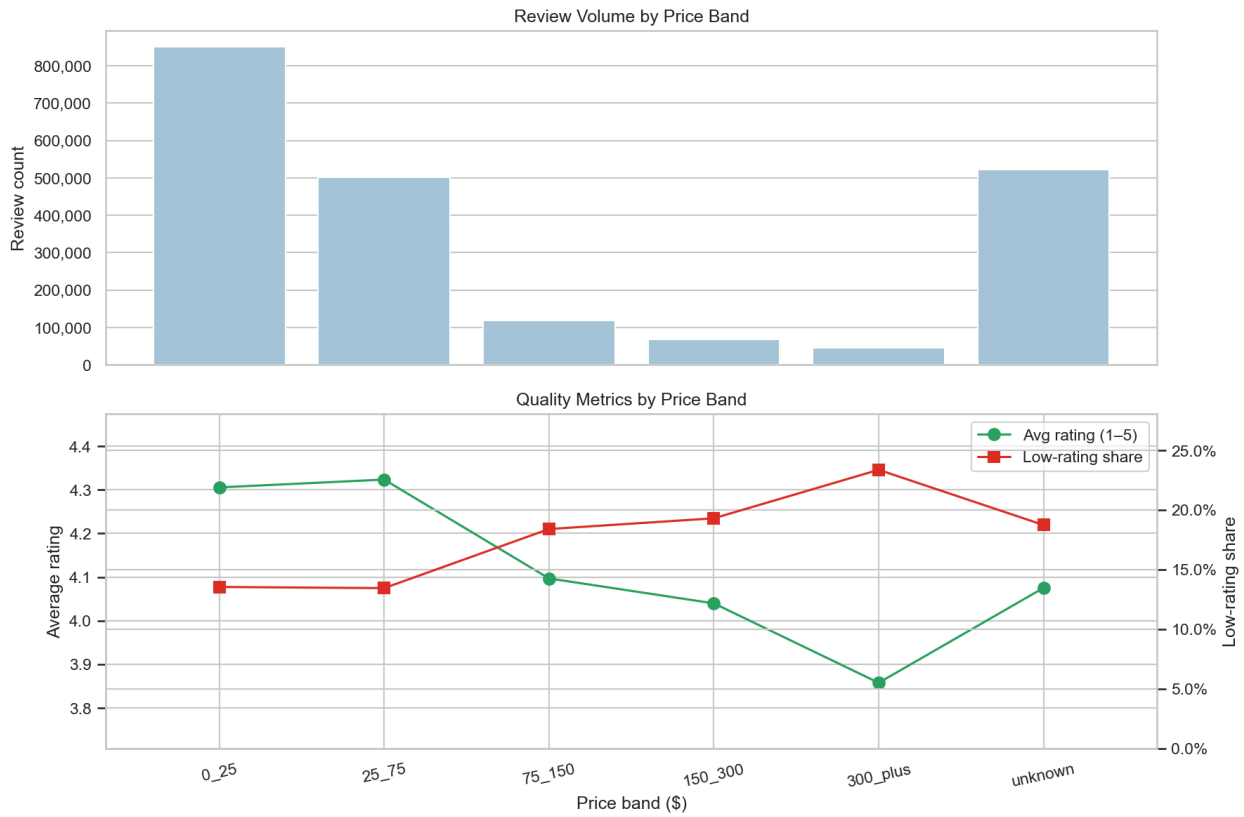


How to read the figure. Horizontal bars order storefronts (`store`) by `low_rating_share` among stores passing the SQL volume threshold. The x-axis is a **percentage** scale.

Takeaways. The longest bars are **seller-level stress** candidates for governance (policy, counterfeit, fulfilment). Pair this chart with `review_count` and `avg_rating` from the underlying CSV so tiny sellers are not over-weighted when counts are low; `avg_helpful_vote` highlights sellers where negative feedback also attracts visibility.

Business conclusion. Long streaks are candidates for audit: listing policy, counterfeiting, delivery times, service. This isn't a "bad store in a vacuum"—you need to look at the volume and average rating from the table to avoid penalizing small sellers for statistical noise. Action: top by low_rating_share + threshold for number of reviews → shortlist for compliance / delisting / negotiations with the marketplace.

7.3.5 Q4 — Price band vs volume and quality



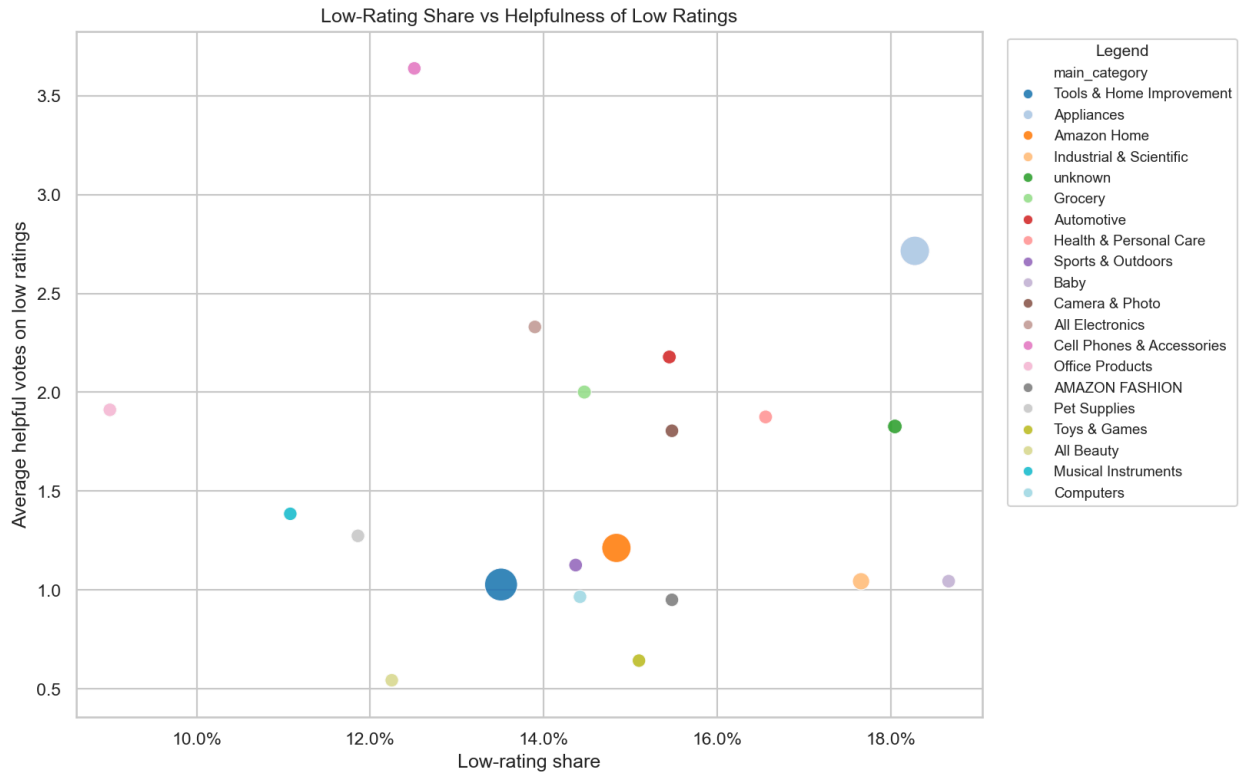
How to read the figure. Upper: `review_count` per `price_band` (volume / exposure). **Lower:** `avg_rating` on the **left** y-axis (green markers) and `low_rating_share` on the **right** y-axis (red markers), so the two quality proxies are **not** forced onto one scale.

Takeaways. Budget bands often concentrate mass-market volume; **premium** bands may show **lower** `avg_rating` and **higher** `low_rating_share`, signalling **value-for-money**, specification accuracy, or support issues worth targeted catalogue review. The `unknown` band can be large — missing price metadata limits pricing conclusions until cleaned.

Business conclusion. Mass-market budget bands often deliver volume, but premium bands can demonstrate a worse expectation-to-actuality ratio (low average rating, higher share of 1–2★)—a typical signal of value-for-money, specifications, warranty, and packaging. A large price uncertainty weakens pricing conclusions—price data needs to be cleaned up. Action: For

assortment and pricing, develop separate playbooks for mass-market and premium bands; close price gaps in the catalog.

7.3.6 Q5 — Negative concentration vs helpfulness on negatives



How to read the figure. Each point is a `main_category` among the **top 20** by `review_count` in this slice. **x-axis:** `low_rating_share`. **y-axis:** `avg_helpful_on_low_ratings`. **Point size** scales with `review_count`; **colour** separates categories.

Takeaways. Categories **far right** carry more severe negativity; categories **high on the y-axis** are those where **critical reviews draw helpful votes** — prioritise remediation where negatives are both frequent **and** influential. Large-volume categories anchor the upper envelope of the cloud; small categories can sit at extremes by chance, so always read **size** together with position.

Business conclusion. To the right – a lot of negativity; up – people actively rate negative content as "helpful," meaning the negative content is noticeable to customers (reputational risk). Categories with a large dot size to the right and up are the first in line for product and content revisions (instructions, packaging, compatibility). Action: prioritize remediation and marketing responses (FAQs, videos, clear specifications) specifically for these quadrants.

8. ML modeling

The objective of Stage III was to implement a robust, end-to-end Machine Learning pipeline optimized for a distributed environment. The implementation focuses on scalability, leveraging Apache Spark on YARN to process the Amazon Reviews dataset. All operations, from feature engineering to model serialization, are performed within the cluster environment, strictly avoiding local execution to maintain architectural integrity.

2. Methodology & Implementation

2.1 Feature Engineering via Hive

A dedicated Hive feature layer was developed to serve as the single source of truth for the ML models.
Implementation: Using stage3_ml_features.hql, a partitioned Parquet table was created. This ensures high-performance data retrieval and efficient column-based storage.
Workflow: The process is triggered via Beeline, ensuring proper integration with the team34_projectdb database.

2.2 Distributed Data Pipeline

The pipeline adheres to the principle of distributed computing by executing all logic on the cluster nodes:
Data Preparation: The stage3_prepare_split.py script utilizes pyspark.ml transformers (StringIndexer, VectorAssembler) to construct feature vectors.
Artifact Generation: Data is partitioned into Training (70%) and Testing (30%) sets. These are stored as JSON artifacts in HDFS, ensuring they are accessible to all Spark executors during training.

2.3 Model Training & Hyperparameter Tuning

Training is implemented in `scripts/stage3_ml_train.py` (invoked from `scripts/stage3.sh`). Two multiclass models are fit with **3-fold cross-validation** and `ParamGridBuilder`:

- **model1 — Random Forest (`RandomForestClassifier`)** on the `features` vector — grid over `maxDepth`, `numTrees`, and related tree hyperparameters.
- **model2 — Multinomial Naive Bayes (`NaiveBayes`)** on `features_nb` — grid over smoothing and model settings appropriate to the discrete feature vector.

3. Technical Compliance & Orchestration

To ensure the pipeline is production-ready for the university cluster, the following constraints were met:

Cluster-Native Execution: Spark ML tasks are submitted via `spark-submit --master yarn`. There is no reliance on local Python for the main ML flow.
Automated Workflow: The Stage III entry point is `scripts/stage3.sh` (Hive feature DDL → prepare/split → `stage3_ml_train.py`). The legacy `scripts/stage3_dummy.sh` only forwards to `scripts/stage3.sh`.
Persistence: Serialized pipelines and train/test artefacts are written under the configured **HDFS** prefixes (for example `project/models/`, `project/data/train`, `project/data/test` when using defaults in the scripts).

4. Deliverables

The following artifacts have been successfully produced:
Evaluation Report: A comprehensive evaluation.csv containing Accuracy, F1-score, Precision, and Recall for all models.
Predictions: Detailed CSV files containing actual labels vs. predictions.
HDFS Storage: Distributed storage of the train.json and test.json datasets and all serialized model pipelines.

5. Label distribution (class imbalance)

Ratings follow the typical Amazon skew: **5★ dominates**, while **1–2★** are comparatively rare. The exported `output/stage3_class_balance.csv` (held-out **test** split, ~30% of labeled rows) illustrates why **accuracy alone is misleading** and why **macro / weighted metrics** (and **confusion matrices**) are essential.

True label	Test count	Approx. share
5★	291,119	**69.6%**
1★	49,425	11.8%
4★	40,680	9.7%
3★	20,510	4.9%
2★	15,560	3.7%

The training split mirrors the same ordering (about **70%** of training rows are **5★** in `stage3_class_balance.csv`). Under imbalance, a naive classifier that **always predicts 5★** would achieve high naïve accuracy — hence we emphasize **F1**, **weighted precision/recall**, and **per-class behaviour** in `evaluation.csv` and in the confusion plots below.

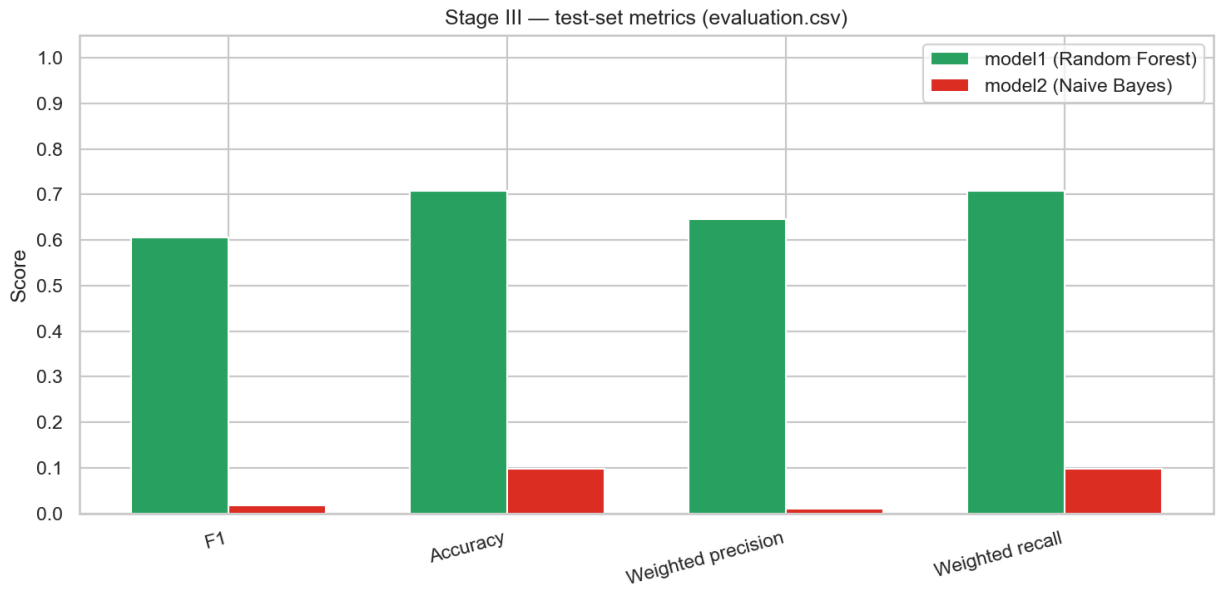
6. Figures and interpretation

Figures are regenerated from `evaluation.csv`, `model1_predictions.csv`, `model2_predictions.csv`, and `output/interpretability/*.csv` using the same plotting logic as `notebooks/stage3_models_plots.ipynb`.

6.1 Terminology (Stage III)

Term	Meaning
model1 / Random Forest	Multiclass classifier on the dense / assembled `features` vector.
model2 / Naive Bayes	Multinomial NB on the discrete `features_nb` subset (different feature space than RF).
F1	Spark pipeline metric on the test partition — primary headline for **multiclass** quality under imbalance.
Weighted precision / recall	Class-frequency–weighted averages (see `evaluation.csv`).
Row-normalized confusion matrix	Each **row** is a **true** star rating; cell values are the **fraction** of that row assigned to each **predicted** rating (per-class error pattern).
Gini importance	Random Forest split-importance ranking (**not** a causal effect size).
`log_prior` **/ priors (NB)**	Training class priors (with smoothing), converted to probabilities for the bar chart.
`log P(feature | class)`	NB multinomial log conditionals for `features_nb` (heatmap).

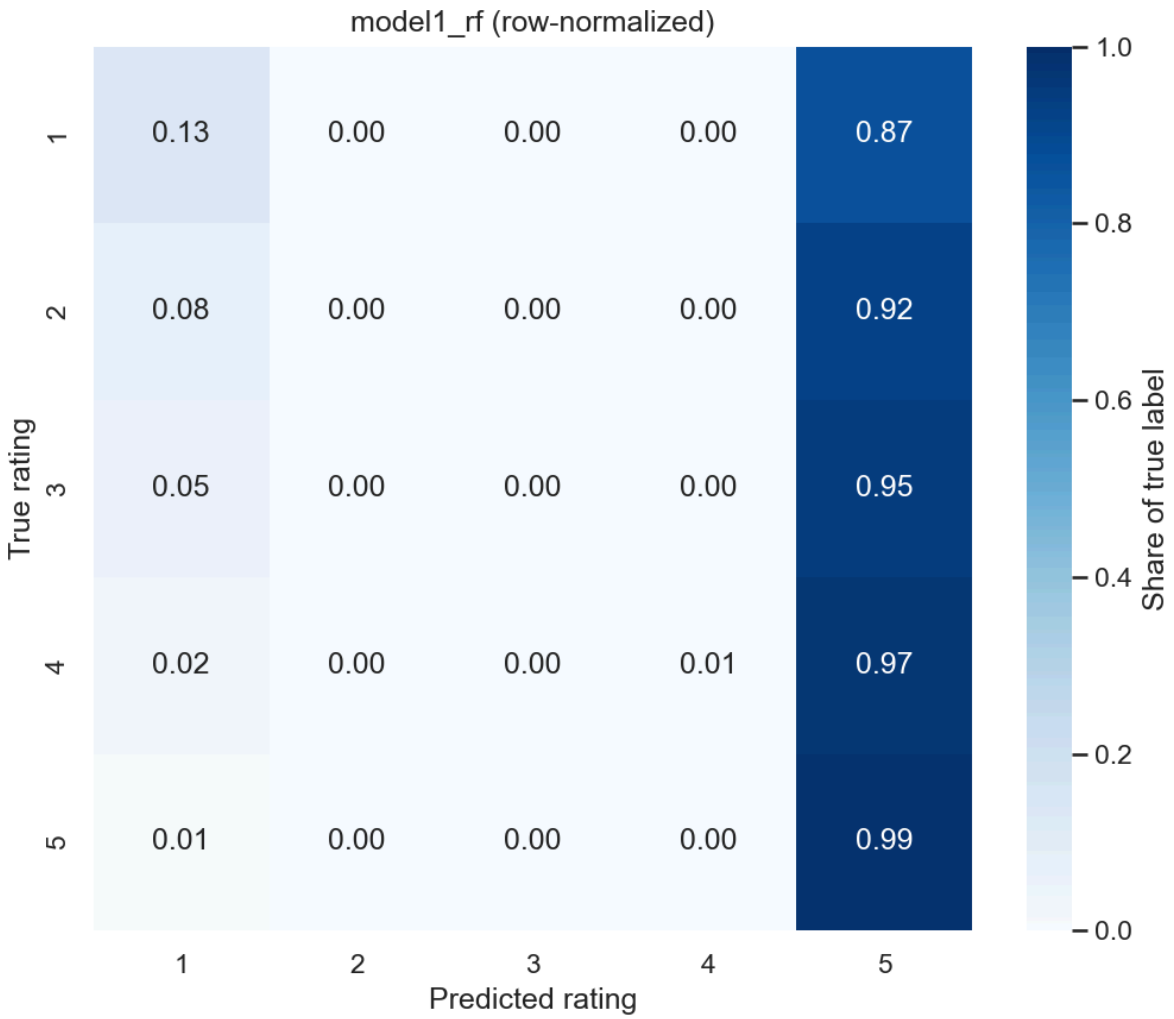
6.2 Test-set metrics (both models)



Numbers (same source as the plot). From `output/evaluation.csv`: **Random Forest** — F1 **~0.605**, accuracy **~0.707**, weighted precision **~0.645**, weighted recall **~0.707**. **Naive Bayes** — F1 **~0.017**, accuracy **~0.097**, weighted precision **~0.009**, weighted recall **~0.097**.

Interpretation. Random Forest exploits non-linear structure and richer inputs; its metrics sit in a **usable** range for this skewed five-class problem. Naive Bayes scores **near random** on headline metrics — consistent with **collapse toward the majority class** when likelihoods are flat on sparse `features_nb` and **priors are dominated by 5★**. **Accuracy** for NB remains **single-digit** despite ~70% of labels being 5★, which underlines that **imbalance does not make NB look "good"** here; the model fails to lock onto the majority vote reliably.

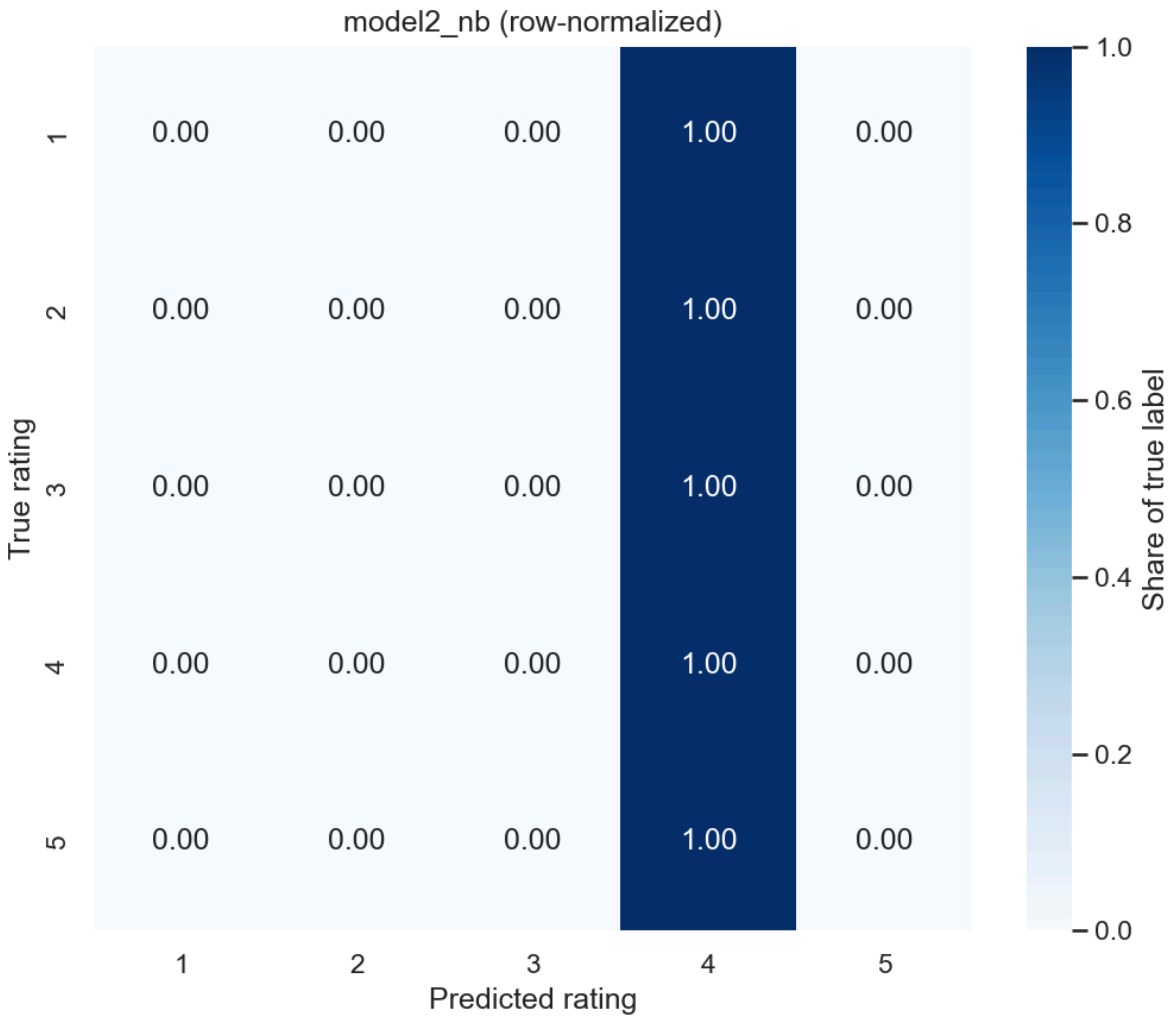
6.3 Random Forest — confusion matrix (row-normalized)



How to read. Rows = true rating, **columns** = predicted rating. Mass **off the diagonal** shows star-level confusion; a **tall column on predicted 5★** would indicate **majority-class collapse** (RF still shows spread, unlike NB).

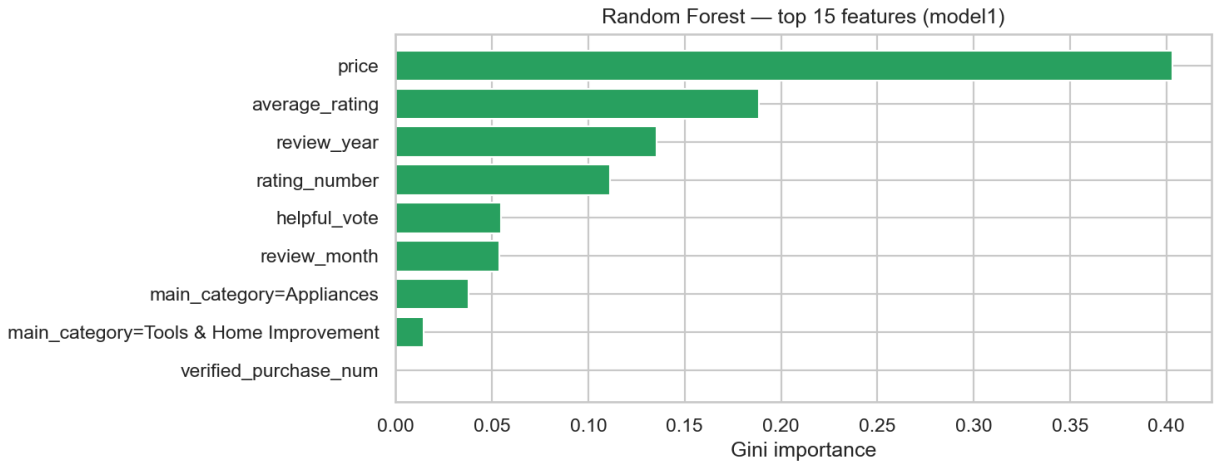
Interpretation. Residual error concentrates on **adjacent stars** (ordinal behaviour). Rare classes (**1–2★**) remain harder — expected under **class imbalance** and fewer training examples. Compare with **§6.5** for which features drive splits.

6.4 Naive Bayes — confusion matrix (row-normalized)



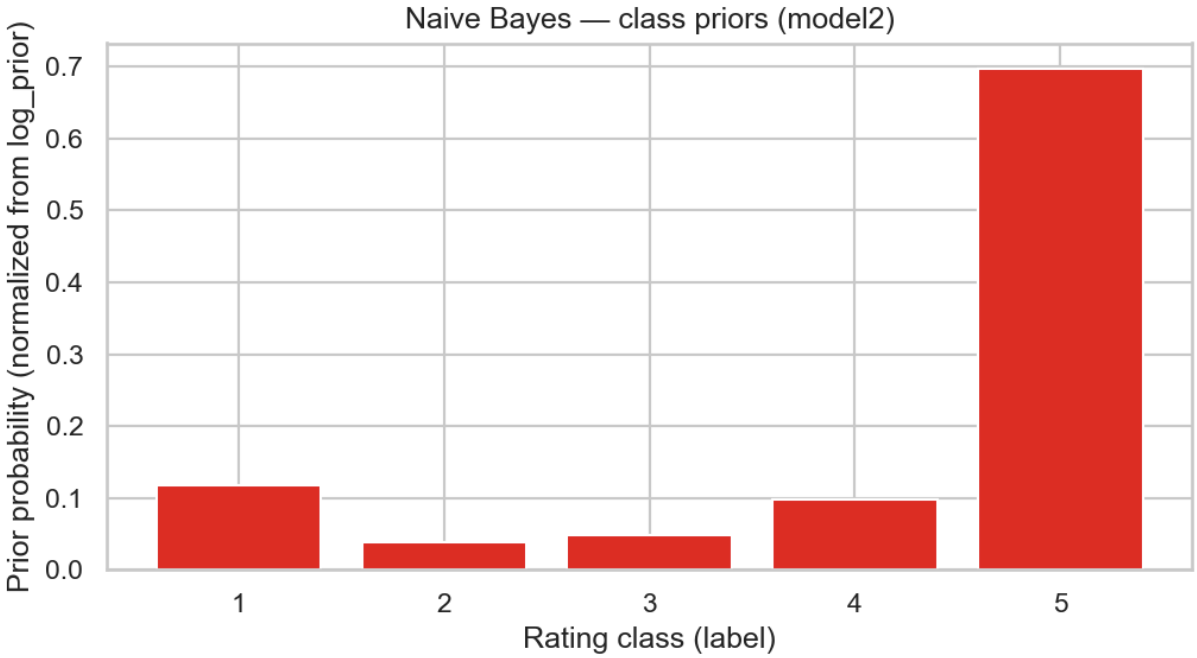
Interpretation. Most probability mass lands in **one predicted column** for many true rows — classic **majority-class / prior-driven** failure when likelihood differences across stars are weak on `features_nb`. This aligns with **very low F1** and shows why **NB is treated as a weak baseline**, not a deployment candidate, on this feature slice.

6.5 Random Forest — feature importance (Gini)



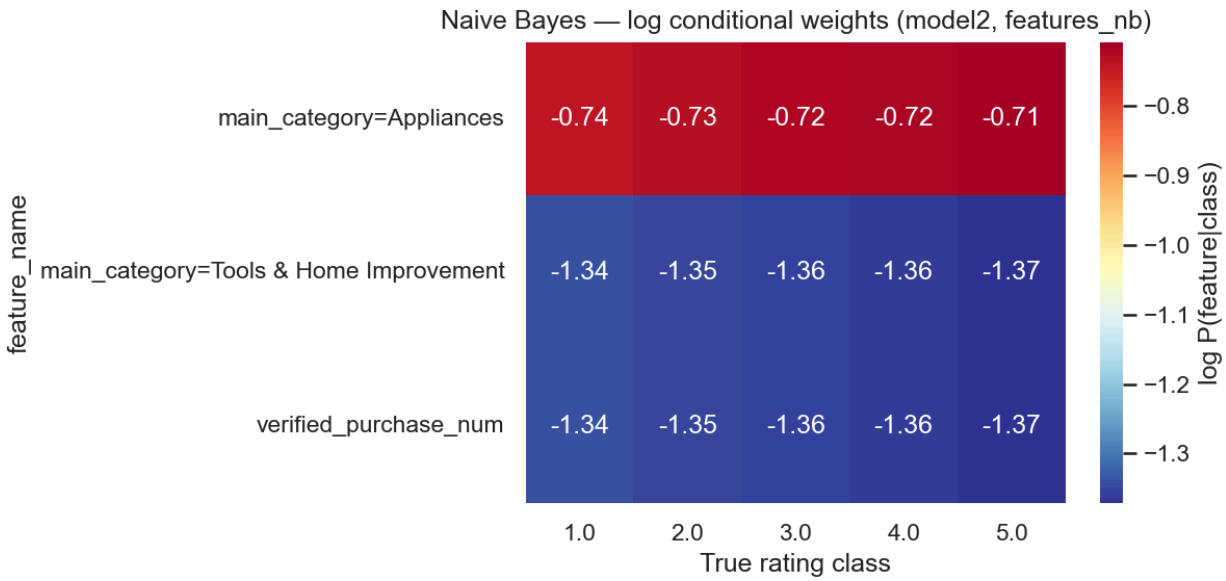
Interpretation. Higher bars reflect **more splitting gain**, not proven causal impact. `price`, `average_rating`, `rating_number`, and calendar fields often rank highly when they **correlate** with the label distribution; **one-hot** `main_category=…` entries capture **category-specific** shifts.

6.6 Naive Bayes — class priors



Interpretation. Priors mirror **training frequency**; the **5★** bucket is largest. A strong prior on **5★** reinforces predicting the modal star when **likelihoods** across classes are similar — contributing to **NB collapse** noted in **§6.4**.

6.7 Naive Bayes — log conditional weights (`features_nb`)



Interpretation. Values are **log P(feature | class)** for the NB feature subset. **Contrast across columns** for the same row (feature) shows where NB thinks a token differs between star labels. **RF still uses the wider `features` vector**, so this heatmap explains **NB behaviour only**.

9. Data presentation (Superset)

Stage IV (**Presentation & Delivery** in the course) requires **Apache Superset**. The checklist expects **EDA and predictive results**, not only raw tables.

9.1 What we could not complete (Superset access)

The team **could not sign in to the course Apache Superset instance with the credentials provided** (login repeatedly failed or access was not granted in time). We **reported the issue to the TA**, but only **about one hour before the presentation**, which left **no practical window** to get replacement credentials, fix account provisioning, or build and **publish** the dashboard. **EDA and ML communication** for grading therefore rely on **this report**, embedded **§§7–8** figures, `output/q*.csv`, and `output/evaluation.csv` / prediction CSVs rather than a live public Superset URL.

10. Conclusion

This project delivers a **batch analytics stack** for the **Amazon Reviews 2023 — Appliances** slice, from raw **JSON Lines** on Hugging Face to **SQL exploration** and **distributed machine learning**. The intended **Superset** narrative (**§9**) could **not** be delivered live because **Superset login with our credentials failed**; see **§9.3**. For grading, **Stage II** and **Stage III** results are reproduced in **§§7–8** and in `output/`

Stage I established a reproducible path: Python ETL builds validated **staging CSV**, **PostgreSQL** holds a normalized **metadata** ↔ **reviews** model with referential integrity, and **Apache Sqoop** lands **Parquet + Snappy** on **HDFS** so later stages read columnar, compressed data with low I/O overhead. Uploading **staging CSV** in parallel preserves a simple audit trail and alternative entry points for tools that do not read Parquet natively

Stage II promoted that lake data into **Hive** structures aligned with analytics: **partitioned** review time (review_year, review_month), **bucketed** layouts on parent_asin, and curated **HiveQL** for category, store, price-band, and time-series views. This made it possible to answer "who, when, and under what product context" questions at scale without repeatedly scanning row-oriented exports

Stage III carried the same architectural principle—**cluster-first execution**—into **PySpark** on **YARN**: feature preparation, train/test materialization on **HDFS**, **3-fold cross-validation** with `ParamGridBuilder` over **two** `pyspark.ml` **model families** (**RandomForestClassifier** as **model1**, **NaiveBayes** as **model2**), comparison on the held-out test split via `output/evaluation.csv`, and persisted pipelines for reproducible evaluation. The emphasis on **YARN** rather than local mode matches the Stage III requirement that ML **not** run in Spark **local** mode

Stage IV was planned as **Apache Superset** (**§9.1**): **PostgreSQL** characteristics, **Stage II** charts with conclusions, and **Stage III** ML panels in **one published dashboard**. In practice, **no published URL** exists because we **could not access Superset** with our credentials and notified TA **only shortly before the demo** (**§9.3**); the same content is documented **inline** in this report

The main residual gap is **causal interpretation**: price, and ratings are **correlational** in observational data; the pipeline is strong for monitoring and prioritization, weaker for claiming that a single lever (e.g. price) *caused* a rating change without controlled experiments

11. Reflections on own work

11.1 Challenges and difficulties

Dataset choice and mid-project pivot. The team initially targeted a different public dataset, but **several groups selected the same source**, which would have made the work hard to distinguish in reviews and increased the risk of accidental overlap in narrative and baselines. To avoid that conflict, the project was **switched to the McAuley-Lab / Amazon-Reviews-2023 — Appliances** slice. The change was the right call for originality, but it cost **time**: ETL field mappings, validation thresholds, report figures, and ML feature assumptions had to be **re-aligned** to the new schema and volumes.

Cluster access from outside Innopolis (e.g. Kazan). The IU Hadoop stack and related services are reachable on the **university network**; from **Kazan**, routine work without the **Innopolis VPN** meant **no reliable path to the cluster** (Hive, HDFS, YARN). That raised the bar for **pair debugging**, lengthened feedback loops, and forced the team to batch cluster runs or rely on teammates who could connect—adding coordination overhead unrelated to the code itself.

Stage IV / Apache Superset. We **never obtained a working Superset login** with the credentials we were given, but **no published dashboard** exists despite prepared data (**§9.3**).

End-to-end environment parity. The same repository must run on **laptops** (often via Dockerized Hadoop/Sqoop), on the **IU shared cluster**, and sometimes in CI-style checks. Small differences—host.docker.internal vs a fixed DB host, **HADOOP_USER_NAME**, or whether sqoop is on PATH—show up only at export time. Keeping **USE_DOCKER_HADOOP**, **HDFS_WAREHOUSE_BASE**, and JDBC URLs in .env documented reduced guesswork but did not remove the need for **trial runs** after each environment change.

Data volume and ETL edge cases. The Appliances split contains on the order of **two million** deduplicated reviews and **tens of thousands** of products. Streaming JSONL to CSV, hashing synthetic **review_id** keys, and enforcing **"review only if metadata exists"** caught orphan rows early but added engineering overhead. **Timestamp semantics** (seconds vs milliseconds in source fields) required explicit handling in Hive so partitions and time trends were not silently wrong.

Hive execution and tooling. Beeline against a remote **HiveServer2** endpoint. Disabling or tuning **vectorized** execution for some Parquet paths, as in stage2_hive_init.hql, was necessary for stability in our stack rather than a generic default.

12. Table of contributions

Column mapping: Member 1 — Dmitriy Ryazanov (data engineer); **Member 2** — Ivan Makarov (ML specialist); **Member 3** — Maksim Ilin (data scientist).

Contribution. Dmitriy Ryazanov — relational schema/migrations alignment with Stage I, Sqoop/HDFS script ownership alongside the team. **Ivan Makarov** — **Stage III** Spark ML implementation (`stage3_ml_train.py`, cross-validation, metric exports). **Maksim Ilin** — **Stage I** execution on the cluster, **Stage II** end-to-end (HiveQL, Beeline, `q1`–`q5`, EDA notebooks), and **Stage III cluster deployment** (`spark-submit` / YARN, HDFS paths, HiveServer2/Metastore connectivity, reproducible runs);

Project tasks	**Task description**	**Dmitriy Ryazanov**	**Ivan Makarov**	**Maksim Ilin**	**Deliverables**	**Avg hours spent**
Stage I — ingestion, warehouse, HDFS	JSONL → staging CSV → validation → migrations/COPY → Sqoop Parquet → HDFS staging. **Primary run & cluster alignment:** M. Ilin.	90%	0%	10%	data/staging/*.csv, migrations/, export/*.sh, `scripts/stage1.sh`, HDFS paths, logs	~60 h
Stage II — Hive & EDA	`stage2_hive_init.hql`, `q1.hql`–`q5.hql`, Beeline, `hive_results.txt`, CSV extracts, EDA notebooks.	0%	0%	100%	`sql/`, `output/q*.csv`, `output/hive_results.txt`, `notebooks/eda_stage2.ipynb`	~30 h

Project tasks	Task description	Dmitriy Ryazanov	Ivan Makarov	Maksim Ilin	Deliverables	Avg hours spent
	Lead: M. Ilin.					
Stage III — Spark ML & cluster	PySpark MLlib on YARN, CV, metrics, interpretability exports. **Training code & grids:** I. Makarov; **cluster rollout & operational runs:** M. Ilin; **support:** D. Ryazanov.	0%	95%	5%	`scripts/stage3*.sh`, `scripts/stage3_ml_train.py`, `output/evaluation.csv`, models on HDFS	~30 h
Stage IV — Superset	Dashboard layout, ≥6 insights + ML panels, publish public URL. **Blocked:** (**§9.3**).	0%	0%	0%		
Documentation & report	README, `docs/*`, `report.md`, `final_report.md`,	90%	5%	5%	report	~30 h

Project tasks	**Task description**	**Dmitriy Ryazanov**	**Ivan Makarov**	**Maksim Ilin**	**Deliverables**	**Avg hours spent**
	Moodle submission.					